

Sprint 1 · "Seguridad y Fundaciones"

Objetivo: Cerrar agujeros de seguridad, crear infraestructura de testing compartida, y que el equipo empiece a testear los módulos más simples del server.

Fernando (Fer) — Tareas principales

T1.1 · Securizar módulo evaluative — eliminar bypass JWT

Prioridad: CRÍTICA · **Estimación:** 4h

El módulo evaluative tiene un bypass JWT para dev que puede llegar a producción. Riesgo de seguridad crítico.

Archivos: `server/src/app/evaluative/`, `server/src/app/auth/`

Acciones:

- Auditar controllers del módulo evaluative
- Implementar flag por `NODE_ENV`: bypass SOLO cuando `NODE_ENV !== 'production'`
- Crear `DevOnlyBypassGuard` que lance excepción en prod
- Revisar `DISABLE_AUTH` para que `MockJwtGuard` no se active en prod

Criterios de aceptación:

- ☐ Ningún endpoint evaluativo accesible sin JWT en producción
- ☐ Test que falle si bypass activo con `NODE_ENV=production`

T1.2 · Auditoría de auth guards en TODOS los controllers

Prioridad: ALTA · **Estimación:** 3h

Garantizar que no hay endpoints expuestos sin autenticación.

Archivos: Todos los `*.controller.ts` en `server/src/app/`

Acciones:

- Verificar guard en cada controller (clase o método)
- Aplicar guards faltantes
- Crear documento `docs/auth-audit.md`

Criterios de aceptación:

- ☐ Todos los controllers protegidos (excepto healthcheck)
- ☐ Tabla de auditoría documentada

T1.3 · Crear `libs/testing/` — utilidades compartidas

Prioridad: ALTA · **Estimación:** 6h

El equipo necesita helpers para escribir tests consistentes. Sin esto, cada uno inventa sus mocks.

Archivos: `libs/testing/` (nueva)

Acciones:

- `createMockPrismaService()` — mock tipado
- `createMockUser()` — factory de `app_user`
- `createMockGameSession()` — factory de sesión
- `createTestingModule()` — helper NestJS con providers comunes
- Coverage thresholds en `jest.config.ts`
- Actualizar CI para enforcement

Criterios de aceptación:

- ☐ `libs/testing/` con los 4 helpers exportados
- ☐ Cl en force a coverage thresholds
- ☐ Documentación de uso en README

● Equipo — Testear código existente

El equipo testea código **existente** sin cobertura. Si encuentran bugs, los arreglan. En la reunión del martes presentan: qué testearon, qué encontraron, qué arreglaron.

Felipe · server/src/app/profile/

Archivos: `profile.controller.ts`, `profile.service.ts`, `dto/updated-preferences.ts` ·

Estimación: 10-12h

Qué hacer:

- Crear specs para controller y service
- Tests: obtener preferencias, actualizar, usuario inexistente
- Si encontrás bugs, arreglalos y documentá qué cambió

Criterios de aceptación:

- ☐ Controller: happypath+usernoencontrado → 404
- ☐ Service: updateprefs, validación DTO
- ☐ Todos en verde

Sofía · server/src/app/auth/

Archivos: `auth.controller.ts`, `jwt-auth.guard.ts`, `jwt.strategy.ts`, `mock-jwt.guard.ts` ·

Estimación: 12-15h

Qué hacer:

- Specs para cada archivo
- Guard permite con token válido, rechaza sin token
- MockJwtGuard inyecta usuario correcto

Criterios de aceptación:

- ☐ Guard: token válido → pasa, sin token → 401
- ☐ MockJwtGuard: inyecta usuario mock
- ☐ Todos en verde

Santiago · server/src/app/game-session/

Archivos: `game-session.service.ts`, `game-session.repository.ts` · **Estimación:** 12-15h

Qué hacer:

- Tests del ciclo: start → attempt → complete
- Sesión ya completada → error
- Sesión inexistente → error

Criterios de aceptación:

- ☐ Service:happypathcompleto
- ☐ Completesesiónyacompletada → error
- ☐ Repository:findByIdinexistente → null
- ☐ Todosenverde

Fernando Gómez · server/src/app/learning/

Archivos: `learning.controller.ts`, `learning.service.ts` · **Estimación:** 10-12h

Qué hacer:

- Tests: bootstrap, crear usuario, crear topic, obtener topics
- Si no valida inputs, agregar validación

Criterios de aceptación:

- ☐ Controller:cadaendpointtienetest
- ☐ Service:happypath+errorpath
- ☐ Todosenverde